Sales Demand Forecasting via Random Forest Regression

A Mathematical Analysis

Author: Jean de Dieu UWINTWALI
UNIVERSITY OF RWANDA
Department: Computer and Software Engineering
Year: 4
Email: uwintwalijeandedieu3@gmail.com

Github Link: https://github.com/UWINTWALI/pyspark-retail-demand-forecasting.git

Jupyter Notebook: https://github.com/UWINTWALI/pyspark-retail-demand-forecasting/blob/main/notebook.ipynb

Abstract

This report describes the development of a system designed to estimate how many products a multinational online retailer is likely to sell in future at a specific date range. The goal is to identify patterns in larger historical sales data and use PySpark, which is built to work efficiently with larger datasets, to fit them in a machine learning model to predict future demand. The analysis uses historical transaction records to examine how features such as time and product categories relate to the number of items sold. The model was trained using 175,452 daily sales records collected between December 2010 and September 2011 and then the model was later evaluated on a separate dataset of 64,524 records. The evaluation shows Mean Absolute Error(MAE) of about 11.5 units, which can be tolerated. Using this model, the anticipated sales for target week 39 of 2011 are estimated at approximately 83,632 units, providing a useful reference for inventory planning and stock management for the e-commerce company.

Contents

Abstract 1
Contents 2
1. Problem Formulation 3
1.1 Formal Setting 3
1.2 Choice of Loss Function 3
2. Data Pipeline and Feature Engineering 4
2.1 Dataset Structure 4
2.2 Temporal Aggregation 4
2.3 Train-Test Split 4
2.4 Feature Engineering 5
2.5 Categorical Encoding via String Indexing 5
2.6 Final Feature Vector 5
3. Regression Trees: Mathematical Foundations 5
3.1 Binary Recursive Partitioning 6
3.2 Impurity Measure for Regression: Variance 6
3.3 The CART Splitting Criterion 6
3.4 Leaf Prediction and Tree Output 6
3.5 Tree Depth and Regularisation 7
4. Random Forest Regression: Theory 7
4.1 Bootstrap Aggregation (Bagging) 7
4.2 Bias-Variance Decomposition 8
4.3 Variance Reduction via Averaging 8
4.4 Decorrelation via Random Feature Subsampling 9
4.5 Effect on Bias 9
5. Model Specification and Implementation 9
5.1 Hyperparameter Configuration 10
5.2 VectorAssembler and Feature Matrix 10
6. Model Evaluation 10
6.1 MAE: Properties and Interpretation 10
6.2 Achieved Results 10
6.3 Week 39 Forecast 11
7. Discussion and Limitations 11
7.1 Model Strengths 11
7.2 Limitations and Future Directions 11

8. Conclusion 12
References..... 12

1. Problem Formulation

Let the universe of retail transactions be described by a dataset D consisting of n transaction records. Each record contains a timestamp, a country identifier, a product stock code, and the quantity of items transacted. After data cleaning and temporal aggregation, we obtain a structured dataset suitable for supervised learning.

1.1 Formal Setting

Definition 1.1 (Input Feature Space)

Let $X = R^d$ denote the input feature space, where $d = 5$ is the number of engineered temporal and categorical features: Day of month, Month, ISO week number, Day of week, and Country index. Each observation x_i in X is a vector

$x_i = (Day_i, Month_i, Week_i, DayOfWeek_i, Country_idx_i)^T$ *in* R^5.

Definition 1.2 (Label Space)

Let $Y = Z \geq 0$ be the space of non-negative integers. The response variable $y_i \in Y$ represents the aggregated quantity of a specific stock code sold in a specific country on a given day.

Definition 1.3 (Regression Problem)

Given a training set $S = \{(xi, yi)\}_{i=1}^{n}$, with $x_i \in X$ and $y_i \in Y$, we seek a measurable function f: X -> R that minimizes the expected prediction error under a chosen loss function

$L: R \times Y \to R^+$.

Specifically, we aim to find the function f* that solves:

$$f^* = \arg\min_{f \in F} E_{(x,y)} \sim P\,[L(f(x), y)],$$

where P is the unknown joint distribution over (X, Y), and F is the hypothesis class of all Random Forest regressors.

1.2 Choice of Loss Function

In this project, model selection and evaluation are performed under the Mean Absolute Error (MAE) loss:

$$L_{MAE}(\hat{y}, y) = |\hat{y} - y|$$

The empirical risk, computed over the test set T $=\{(xj, yj)\}_{j=1}^{m}$, is:

$$\mathbf{MAE} = \frac{1}{m} \sum_{j=1}^{m} |f(x_j) - y_j|$$

Remark: *MAE is preferred over Mean Squared Error (MSE) in this context because retail demand data exhibits heavy-tailed distributions with occasional bulk orders. MAE is more robust to such extreme values since it does not square the residuals and therefore does not disproportionately penalize large single-transaction deviations.*

2. Data Pipeline and Feature Engineering

Before any modelling can occur, the raw transaction-level data must be transformed into a well-structured feature matrix. This section formalizes each step of that transformation.

2.1 Dataset Structure

The raw dataset contains 384,721 transaction records across 13 attributes or columns. We retain four primary columns for our machine learning modelling: InvoiceDate (t), Country (c), StockCode (s), and Quantity (q).

2.2 Temporal Aggregation

Since multiple transactions for the same product and country may occur on a single day, we aggregate by summing quantities:

$$Q(t, c, s) = \sum_{\substack{k:\, date_k, \\ Country_k, \\ Stock_k = S}} q_k$$

This reduces the 384,721 raw records to 239,976 daily (date, country, stockCode) combined. The aggregation is formally a groupBy operation under the equivalence relation (t, c, s), followed by summation of the Quantity column within each equivalence class.

2.3 Train-Test Split

The dataset is partitioned chronologically at the boundary date t* = 2011-09-25:

$$S_{train} = \{(x_i, y_i) : date_i \le t^*\}, \quad |S_{train}| = 175{,}452$$

$$S_{test} = \{(x_j, y_j) : date_j > t^*\}, \quad |S_{test}| = 64{,}524$$

Instead of shuffling all the data together and randomly picking which goes into training versus testing, we kept everything in chronological order. The model was trained on older sales data up to September 2011 and tested on newer data after that date. This mimics how the system would work in the real world and prevent the leakage (model must not learn from training test)

2.4 Feature Engineering

From the date column t, we extract four scalar temporal features using standard calendar arithmetic. Let Year(t), Month(t), Day(t), ISOWeek(t), and DayOfWeek(t) denote the respective calendar components of date t.

The feature extraction map φ: Date -> R^4 is defined as:

$$\varphi(t) = (\ \mathrm{Day}(t),\ \mathrm{Month}(t),\ \mathrm{ISOWeek}(t),\ \mathrm{DayOfWeek}(t)\ )$$

where DayOfWeek(t) in {1, 2, ..., 7} following ISO convention (1 = Monday). These features encode seasonality at multiple granularities: daily cycles (DayOfWeek), monthly patterns (Day, Month), and ISO weekly patterns (ISOWeek).

2.5 Categorical Encoding via String Indexing

Definition 2.1 (Frequency-Ordered Label Encoding)
Let $V = \{v_1, v_2, ..., v_k\}$ be the vocabulary of a categorical feature with K distinct values. The StringIndexer assigns a numeric index $\sigma(v_k)$ in {0, 1, ..., K-1} to each value according to descending frequency of occurrence in the training corpus. That is, if $count(v_i) \geq count(v_j)$ in S_{train}, then $\sigma(v_i) \leq \sigma(v_j)$.

In practice, for the Country feature with K_c distinct countries, we obtain Country_idx in $\{0, 1, ..., K_c - 1\}$. The most frequent country, United Kingdom, receives index 0.

Remark: *The StockCode feature was excluded from the feature vector on purpose. Although it has 3,616 unique values in the training set the numbers don't imply any kind of ranking and doesn't contribute to model performance. If we tried to convert each product code into a number, we'd end up with 3,616 separate numerical columns that don't actually mean anything useful, they'd just be random labels with no real relationship to sales patterns. Including it would inflate model complexity without adding predictive signal about seasonality, promotion cycles, or demand patterns.*

2.6 Final Feature Vector

The complete feature construction map Φ: (Date, Country) -> R^5 is:

$$\Phi(t, c) = (\ \mathrm{Day}(t),\ \mathrm{Month}(t),\ \mathrm{ISOWeek}(t),\ \mathrm{DayOfWeek}(t),\ \sigma(c)\ )$$

Each training example is therefore the pair $(\Phi(t_i, c_i), Q(t_i, c_i, s_i))$.

3. Regression Trees: Mathematical Foundations

The Random Forest model is an ensemble of individual regression trees. We first establish the mathematical framework for a single regression tree, as this is the fundamental building block.

3.1 Binary Recursive Partitioning

Definition 3.1 (Axis-Aligned Partition)
A regression tree partitions the feature space $X = R^d$ into M disjoint rectangular regions R_1, R_2, ..., R_M such that $R_i \cap R_j = \emptyset$ for $i \neq j$, and $\cup_{m=1}^{M} R_m = X$.

Each internal node of the tree applies a binary split of the form: for some feature index j in {1,...,d} and threshold τ in R, the node routes an input x to the left child if $x_j \leq \tau$, and to the right child if $x_j > \tau$. A leaf node m outputs a constant prediction c_m.

3.2 Impurity Measure for Regression: Variance

For regression tasks, the natural impurity measure at a node is the variance of the response values in that node. Given a set of observations I at a node, the impurity is:

$$\widehat{Var}(I) = \left(\frac{1}{|I|}\right) \sum_{i \in I} (y_i - \bar{y}I)^2$$

Where $\bar{y}I = \left(\frac{1}{|I|}\right) \sum_{i \in I} y_i$ is the sample mean of the response in that node.

3.3 The CART Splitting Criterion

The Classification and Regression Trees (CART) algorithm greedily selects the split (j*, τ*) that maximally reduces variance. At a node containing observation set I, for each candidate feature j and threshold τ, define the left and right child sets:

$$I_L(j, \tau) = \{ i \in I : x_{ij} \leq \tau \}, \quad I_R(j, \tau) = \{ i \in I : x_{ij} > \tau \}$$

The optimal split solves:

$$(j^*, \tau^*) = \arg\min_{j,\tau} \frac{[\, |IL| * Var(I_L(j,\tau)) + |I_R| * Var(I_R(j,\tau)) \,]}{|I|}$$

This is equivalent to maximising the weighted variance reduction:

$$\Delta Var(j, \tau) = Var(I) - \left[\frac{(|I_L|}{|I|} * Var(I_L) + \frac{|IR|}{|I|} * Var(I_R) \right]$$

Remark: *Since Var(I) is constant with respect to the split (j, τ), minimizing the weighted sum of child variances is exactly equivalent to maximizing the variance reduction. The CART algorithm proceeds recursively until a stopping criterion is met, such as a maximum tree depth or a minimum node size.*

3.4 Leaf Prediction and Tree Output

At each leaf node m, the prediction is the conditional mean of the training responses in that region:

$$c_m = \bar{y}R_m = \frac{1}{|\{i: x_i \in R_m\}|} * \sum_{i:\, x_i \in R_m} y_i$$

The full tree prediction for a new input x is therefore:

$$T(x) = \sum_{m=1}^{M} c_m \;.\; 1[\, x \in R_m]$$

Each tree is thus a piecewise-constant function on X.

Optimality of Conditional Mean at Leaf

For any leaf region R_m, the prediction $c_m = \bar{y}R_m$ is the minimizer of the squared error loss within that region.

Formally, $c_m = \arg\min_{c \in R} \sum_{i:\, x_i \in R_m} (y_i - c)2$.

Proof: Let $L(c) = \sum_{i \in R_m} (y_i - c)^2$. Differentiating with respect to c and setting to zero:

$$\frac{dL}{dc} = \sum_{i \in R_m} -2(y_i - c) = 0$$

$$\Rightarrow c = \frac{1}{|R_m|} \sum_{i \in R_m} y_i = \bar{y}R_m$$

The second derivative $\frac{d^2L}{dc^2} = 2 \,.\, |R_m| > 0$, this critical point is a global minimum.

3.5 Tree Depth and Regularisation

In this project, trees are grown to a maximum depth of maxDepth = 10. This means no root-to-leaf path may contain more than 10 splits, bounding the complexity of each tree. Additionally, maxBins = 50 controls the discretisation of continuous features when searching for optimal splits.

The depth constraint serves as structural regularization: shallower trees have higher bias but lower variance. The ensemble averaging in the Random Forest counteracts the increased bias while further reducing variance, as formalized in Section 4.

4. Random Forest Regression: Theory

A Random Forest is a specific instantiation of Bootstrap Aggregation (Bagging) applied to decision trees, with an additional source of randomisation in the feature selection at each split. We now derive the core theoretical properties of this method.

4.1 Bootstrap Aggregation (Bagging)

Definition 4.1 (Bootstrap Sample)

Given a training set $S = \{(xi, yi)\}_{i=1}^{n}$, a bootstrap sample S^* is obtained by drawing n samples uniformly at random with replacement from S. Each bootstrap sample has the same size as S, but some observations appear multiple times while others are omitted.

The probability that a specific observation (x^i, y^i) is NOT included in a given bootstrap sample is:

$$P(i \notin S^*) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \to \infty} e^{-1} \approx 0.368$$

Thus each bootstrap sample uses approximately 63.2% of the training observations. The remaining ~36.8% are called Out-of-Bag (OOB) observations and can be used for unbiased error estimation.

Definition 4.2 (Bagged Regressor)
Given B bootstrap samples $S^*_1, S^*_2, \ldots, S^*_B$, the Bagged predictor is the average of B base estimators T_b trained independently on each bootstrap sample:

$$f_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

4.2 Bias-Variance Decomposition

The expected prediction error of any regressor f at a point x can be decomposed into three components. Assuming the true relationship is y = f*(x) + epsilon where epsilon ~ (0, σ^2_epsilon):

$$E[(y - f(x))^2] = Bias^2(f(x)) + Var(f(x)) + \sigma_\varepsilon^2$$

where:

$$Bias(f(x)) = E[f(x)] - f^*(x)$$

$$Var(f(x)) = E[\,(f(x) - E[f(x)])^2\,]$$

The irreducible error σ_ε^2 cannot be reduced by any model. The goal is therefore to minimize $Bias^2$ + Var jointly.

4.3 Variance Reduction via Averaging

Theorem 4.1 (Variance Reduction by Bagging)
Suppose $T_1, T_2, \ldots, T_B$ are identically distributed base estimators, each with variance $\sigma^2 = Var(T_b(x))$, and with pairwise correlation **rho** = $corr(T_a(x), T_b(x))$ for a ≠ b. Then the variance of the bagged predictor $f_{bag}(x) = \frac{1}{B}\sum_b T_b(x)$ is:

$$Var(f_{bag}(x)) = rho \times \sigma^2 + (1 - rho) \times \frac{\sigma^2}{B}$$

Proof: Using the linearity of variance for the average of B correlated variables:

$$Var(f_{bag}) = \frac{1}{B^2} \times Var(\sum_{b=1}^{B} T_b(x))$$

$$= \frac{1}{B^2} \times [\,B\sigma^2 + B(B-1) \times rho \times \sigma^2\,]$$

$$= \frac{\sigma^2}{B} + \frac{(B-1)}{B} \times rho \times \sigma^2$$

$$= rho * \sigma^2 + (1 - rho) \times \frac{\sigma^2}{B}$$

Key observations from this result:

(1) As $\xrightarrow{B\to\infty}$, the variance approaches rho × σ^2, the irreducible floor imposed by between-tree correlation.

(2) When **rho = 0** (independent trees), $Var(f_{bag}) = \frac{\sigma^2}{B}$, which decreases without bound as B grows.

(3) The bagged predictor always has lower variance than any single tree, regardless of B, provided **rho** < 1.

4.4 Decorrelation via Random Feature Subsampling

What makes the Random Forest different from basic Bagging is that it uses random feature subsampling at each split. At every split in every tree, a random subset of m_{try} features is drawn uniformly without replacement from all d = 5 features, and the best split is found only within this subset.

In Bagging on the full training data, trees grown on different samples of the date tend to be dominated by the same highly predictive features (For example, Month and ISOWeek in seasonal sales data). This induces positive correlation **rho > 0** among the trees, and hence this limits the variance reduction in Theorem 4.1.

By forcing each split to only consider a random few features(m_{try} subset), Random Forests artificially reduce the similarity between trees. If we label the correlation between trees as $\boldsymbol{rho}_{\mathbf{RF}}$, one obtains:

$$\boldsymbol{rho}_{\mathbf{RF}} < \boldsymbol{rho}_{\mathbf{Bagging}}$$

Thus, by Theorem 4.1, $\mathrm{Var}(f_{\mathrm{RF}}) < \mathrm{Var}(f_{\mathrm{Bagging}})$. The common default is $m_{\mathrm{try}} = \mathrm{floor}(\frac{d}{3})$ for regression, though PySpark's RandomForestRegressor defaults to$m_{\mathrm{try}} = \mathrm{floor}(\frac{d}{3}) = 1$ when d = 5. The 100 trees used in this project (numTrees = 100) ensure B is sufficiently large to closely approach the asymptotic lower bound $\boldsymbol{rho}_{\mathbf{RF}} \times \boldsymbol{\sigma}^2$.

4.5 Effect on Bias

Theorem 4.2 (Bias Invariance of Bagging)

The average prediction from bagged model equals the average value of a single tree. Consequently, thuis means that bagging does not add any extra bias:

$$\mathbf{E}[\mathbf{f}_{\mathbf{bag}}(\mathbf{x})] = \frac{1}{\mathrm{B}} \sum_{b=1}^{B} \mathbf{E}[\boldsymbol{T}_{\boldsymbol{b}}(\mathbf{x})] = \mathbf{E}[\mathbf{T}_{\mathbf{b}}(\mathbf{x})]$$

Proof: By linearity of expectation and the assumption that all T_b are identically distributed:

$$\mathbf{E}[\mathbf{f}_{\mathbf{bag}}(\mathbf{x})] = \mathbf{E}[\frac{1}{\mathrm{B}} \sum_{b=1}^{B} \boldsymbol{T}_{\boldsymbol{b}}(\mathbf{x}) = \frac{1}{\mathrm{B}} \sum_{b=1}^{B} \mathbf{E}[\mathbf{Tb}(\mathbf{x})] = \mathbf{E}[\mathbf{T}_{\mathbf{1}}(\mathbf{x})]$$

This is a critical property: the Random Forest reduces variance without introducing any additional bias relative to a single tree. So as you add extra trees(as B increases.), the overall error goes down.

5. Model Specification and Implementation

The model is implemented in Apache Spark MLlib using PySpark. Spark processes data as distributed DataFrames and applies the ML pipeline in a scalable, fault-tolerant manner, making it suitable for the scale of this larger dataset of 384,721 records.

5.1 Hyperparameter Configuration

Hyperparameter	Value	Justification
numTrees (B)	100	Approaches asymptotic variance floor **rho × σ^2**
maxDepth	10	Balances model expressiveness and overfitting risk
maxBins	50	Granularity of threshold search for continuous splits
d (features)	5	Day, Month, Week, DayOfWeek, Country_idx
random seed	42	Ensures reproducibility of bootstrap samples

5.2 VectorAssembler and Feature Matrix

PySpark's VectorAssembler concatenates the d = 5 scalar feature columns into a single dense feature vector for each observation. For the ith observation, this produces:

$$\mathbf{x_i = [Day_i, Month_i\ Week_i, DayOfWeek_i, Country_idx_i]^T \in R^5}$$

The training data is thus represented as a matrix X_{train} in $R^{175452 \times 5}$ and a label vector $y_{train} \in Z^{175452.}$

6. Model Evaluation

6.1 MAE: Properties and Interpretation

The Mean Absolute Error is the empirical estimate of E[|f(X) - Y|] under the test distribution. It is computed as:

$$\mathbf{MAE} = \frac{1}{m}\sum_{j=1}^{m} |\mathbf{f}(\mathbf{x}_j) - \mathbf{y}_j|$$

MAE has several desirable mathematical properties for this application:

i. Scale equivariance: MAE(cf, cY) = c . MAE(f, Y) for any positive constant c.
ii. Robustness: Unlike MSE, MAE is not affected by outlier squared growth. Its influence function is bounded.
iii. Units: MAE is expressed in the same units as the response variable (units of product sold), making it directly interpretable by business stakeholders.
iv. Subgradient: The absolute value loss is convex but not differentiable at zero; its subgradient is sgn($\hat{y}$ - y), which lies in [-1, 1].

6.2 Achieved Results

On the held-out test set S_{test} (64,524 records covering October to December 2011):

MAE = 11.498 units per product per day

This means that on average, the model's prediction of the daily quantity sold for a given (Country, StockCode) pair deviates from the actual value by approximately **11.5 units.**

Given that the mean transaction quantity in the dataset ranges from small single-unit purchases to bulk orders of several hundred units, this error is within an operationally acceptable range.

6.3 Week 39 Forecast

Week 39 of 2011 corresponds to the calendar dates 26 September up to 2 October 2011. These dates fall within the test period (after the split date of 25 September 2011). The model generates per-record predictions $f(x_j)$ for every (Date, Country, StockCode) tuple in Week 39, and the aggregate forecast is:

$$\widehat{Q}\{\text{week39}\} = \sum_{\{j:\ \text{ISOWeek}(\text{date}_j)=39\}} f(x_j) = 83{,}632 \ \ \text{units}$$

This figure is obtained by summing the model's continuous predictions over all records in Week 39 and converting to an integer. The forecast signals a seasonal uptick consistent with early pre-holiday shopping activity, providing the Sales and Operations team with a concrete basis for stock replenishment decisions.

7. Discussion and Limitations

7.1 Model Strengths

- Non-parametric flexibility: The Random Forest makes no distributional assumption about the response variable. It can model non-linear interactions between features such as the joint effect of Month and DayOfWeek on purchasing behaviour.
- Parallelisability: Each tree in the ensemble is trained independently on its bootstrap sample, enabling trivial data-parallel computation in Apache Spark.
- Built-in feature importance: The Random Forest computes a natural measure of variable importance based on the cumulative variance reduction attributable to each feature across all splits in all trees.

7.2 Limitations and Future Directions

- Stationarity assumption: The model implicitly assumes that the temporal patterns learned from the training period (December 2010 to September 2011) generalise to the test period. A structural break (e.g., a new product launch or an economic shock) would violate this assumption.
- Product-level granularity: StockCode was excluded to avoid high-cardinality encoding issues. Future work could incorporate product embeddings or aggregate product-level historical demand statistics (e.g., rolling mean, trend) as additional features.
- No explicit seasonality model: Techniques such as Fourier features or Gaussian Process priors over time could provide a more principled probabilistic model of demand seasonality.

- Interval forecasting: The current model produces point estimates. Conformal prediction or quantile regression forests could extend this to prediction intervals, allowing the S&OP team to reason about demand uncertainty.

8. Conclusion

This report has developed a complete mathematical framework for sales demand forecasting using Random Forest Regression. Starting from a formal regression problem statement, we derived the CART variance-reduction splitting criterion, proved the leaf-mean optimality theorem, and established the bias-variance decomposition that underpins the ensemble's theoretical advantage. Theorem 4.1 shows that the variance of the ensemble predictor is bounded above by **rho × σ^2**, where rho is the inter-tree correlation and σ^2 is the variance of a single tree; Theorem 4.2 confirms that this variance reduction is achieved without any bias penalty.

The implemented model achieves a **MAE** of **11.50 units** on 64,524 held-out test records and forecasts 83,632 units to be sold in ISO Week 39 of 2011. These results demonstrate that a rigorously motivated ensemble of Random Forest trees, applied to well-engineered temporal and categorical features and distributed via Apache Spark, constitutes an effective and scalable approach to e-commerce demand forecasting.

References

[1] Breiman, Leo, et al. *Classification and Regression Trees*. Routledge, 19 Oct. 2017, doi.org/10.1201/9781315139470.

[2] Efron, B. "Bootstrap Methods: Another Look at the Jackknife." *The Annals of Statistics*, vol. 7, no. 1, Jan. 1979, pp. 1–26, https://doi.org/10.1214/aos/1176344552.

[3] Geurts, Pierre, et al. "Extremely Randomized Trees." *Machine Learning*, vol. 63, no. 1, 2 Mar. 2006, pp. 3–42, orbi.uliege.be/bitstream/2268/9357/1/geurts-mlj-advance.pdf, https://doi.org/10.1007/s10994-006-6226-1.

[4] Hastie, Trevor, et al. *The Elements of Statistical Learning*. *Springer Series in Statistics*, New York, NY, Springer New York, 2009.

[5] https://plus.google.com/u/0/+Datacamp. "Get Started." *DataCamp*, 2026, app.datacamp.com/learn/projects/2144. Accessed 9 Mar. 2026.

[6] Spark, Apache . "Classification and Regression - Spark 2.4.5 Documentation." *Spark.apache.org*, spark.apache.org/docs/latest/ml-classification-regression.html.

[7] Breiman, Leo. "Random Forests." *Machine Learning*, vol. 45, no. 1, Oct. 2001, pp. 5–32.

[8] Breiman, Leo. "Bagging Predictors." *Machine Learning*, vol. 24, no. 2, Aug. 1996, pp. 123–140, link.springer.com/article/10.1007/BF00058655.